Week 2: Introduction to Claude

53

Neil Cherry, BS-CS
Joseph E. Wilkes, PhD-EE

September 2026

Questions from Last Week

Some Popular AI systems

- Copilot
- Cursor
- Codex
- Claude Code
- Gemini CLI
- Local models

- This class will focus on Claude Code
- As of the creation of this class, Claude was considered the best for code development
- But, things change fast, so the future may be different
- Neil found CoPilot troublesome so we did not include it
- Many of the techniques we will learn apply to other AIs
- We will provide a comparison of these 6 approaches in week 4

Claude AI (claude.ai)

- Best features:
  - Code development and testing
  - Web development
  - Document development
- How to interact with Claude:
  - Tokens
  - Markdown language/Documents
  - Claude Prompts
  - Claude Skills
  - Claude Tools
  - MCP

Before We Start:

“The Last Question” a Short Story by Isaac Asimov

- **The Beginning:** In 2061, two technicians ask the supercomputer Multivac if the sun and stars can be kept from burning out. Multivac answers: *"There is as yet insufficient data for a meaningful answer.”*
- **The Passage of Time:** Across multiple future eras and galactic expansions, humans ask upgraded versions of the computer the same question about entropy, always receiving that same reply.
- **The End of the Universe:** Eventually, humanity merges into a single cosmic consciousness linked to the final form of the computer in hyperspace, and physical time ends as the stars die out.
- **The Conclusion:** Left alone in the dark void, the computer finally figures out how to reverse entropy, and with no one left to tell, it demonstrates the solution by declaring, *"LET THERE BE LIGHT!"* and birthing a new universe.

Tokens

- Claude charges Tokens for each action it takes
- Discovering Token usage limits is difficult, there are many variables
- Claude Pro ($20/mo): Provides access to high-capacity usage but is subject to activity limits.
- Claude Max Plans:
  - Max5: Approx. 88,000 tokens per 5-hour window.
  - Max20: Approx. 220,000 tokens per 5-hour window.
- 5-Hour Rolling Reset: Limits reset every 5 hours, rather than once per day, often based on when the first message in a session was sent.
- Context Window: Most paid plans have a 200,000 token context window, allowing large prompt inputs, though this consumes the 5-hour limit faster.
- Users have found tricks to get maximum effectiveness before hitting token limits
  - One trick is to start a 5 hour window with a simple request,
  - Do some other work outside of Claude and then do the token expensive work just before the 5 hour window expires,
  - Then roll into a new 5 hour window to finish the work.

https://support.claude.com/en/articles/11647753-how-do-usage-and-length-limits-work

Token Limits according to Claude

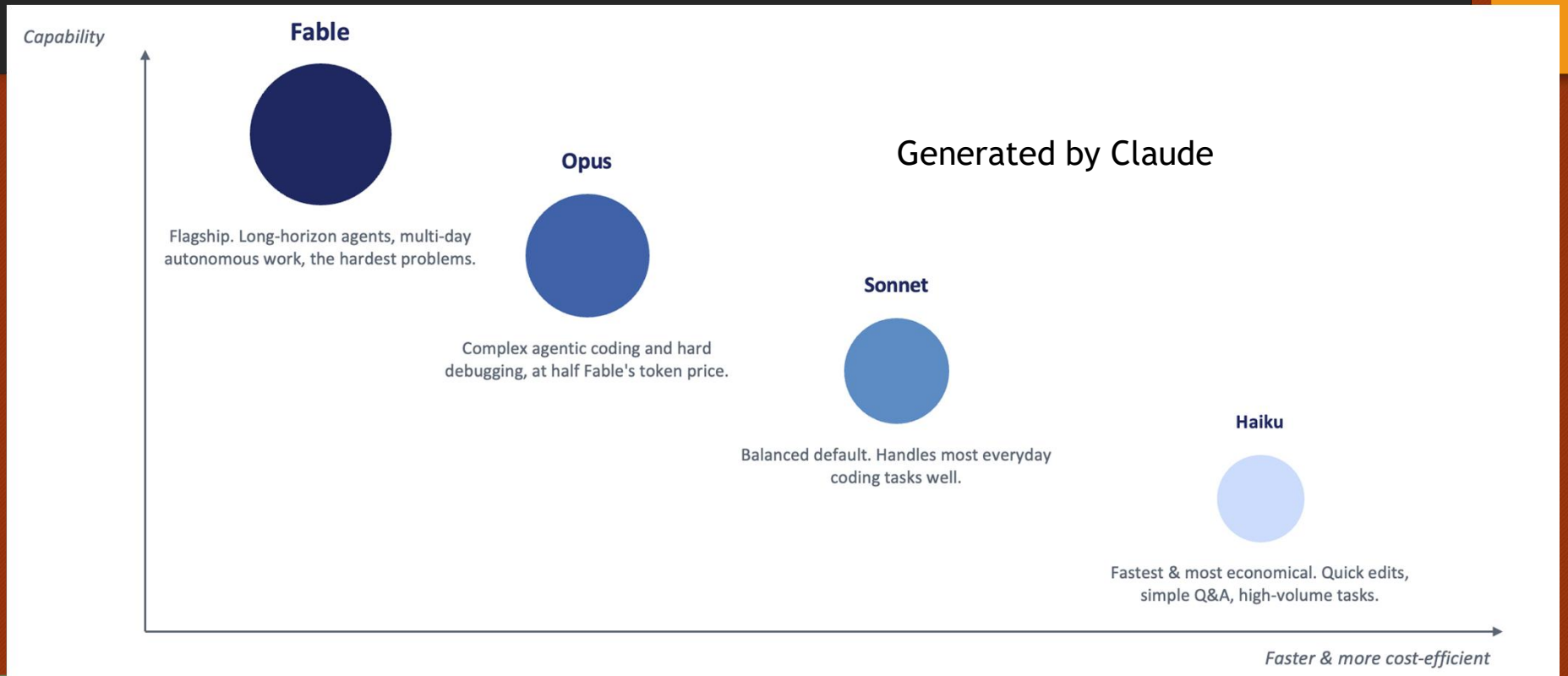
- Claude accounts have two different kinds of limits:
  - **Context window (per-conversation token limit)** — This is the closest thing to a strict "token limit." Across all paid plans (Free, Pro, Max, Team), Claude's context window is **200K tokens**. Enterprise plans get **500K tokens** on some models. This is the working memory for a single conversation — input, attached files, and ongoing exchanges all count against it. When you approach the cap, Claude (with code execution enabled) auto-summarizes earlier messages to keep going.
  - **Usage limits (rate-based, across conversations)** — Anthropic does not publish hard per-plan token numbers for these. Instead they describe it as a "conversation budget" that resets every **5 hours** for session limits, with Pro/Max also subject to a **weekly limit** that resets 7 days after a session starts.
- How much you actually get depends on:
  - Your plan (Free < Pro < Max < Team < Enterprise)
  - Message/conversation length and attachments
  - Which model you use (Opus burns through limits faster than Sonnet/Haiku)
  - Which features are on (extended thinking, web search, MCP connectors, projects all consume more)
- Usage across claude.ai, Claude Code, and Claude Desktop all counts toward the same pool
- If you hit a usage limit you can wait for the reset, upgrade, or buy extra usage.
- Enterprise has a separate "usage-based" billing tier where you pay for consumption directly rather than fitting under a fixed allowance.
- For Claude Code specifically, this article has model-by-model detail: https://support.claude.com/en/articles/14552983-models-usage-and-limits-in-claude-code

CDL
INFOAGE SCIENCE AND HISTORY MUSEUMS

Claude has multiple tiers of reasoning

- As reasoning becomes deeper, Claude spends more time and tokens
- Depending on the work, you many need to change the level selected
- The following slide generated by Claude shows the difference between tiers

Choosing a Model Tier



Claude Prompts

- Prompts are the language that is used to talk to Claude and tell it what you want to do
- The document on ”Prompting best practices” is very long
  - https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices
- The document covers general approaches, coding, and documents
- Read the entire document to gain an fuller understanding of Claude Prompts
- The following slides summarize some important points
- Like learning any new language, you must practice it and learn how to communicate

Prompts: Calibrating Effort and Thinking Depth - 1

- **The effort parameter allows you to tune Claude's intelligence vs. token spend, trading off capability for faster speed and lower costs**
  - **max**: Max effort can deliver performance gains in some use cases, but may show diminishing returns from increased token usage. This setting can also sometimes be prone to overthinking. We recommend testing max effort for intelligence-demanding tasks.
  - **xhigh** (new): Extra high effort is the best setting for most coding and agentic use cases.
  - **high**: This setting balances token usage and intelligence. For most intelligence-sensitive use cases, we recommend a minimum of high effort.
  - **medium**: Good for cost-sensitive use cases that need to reduce token usage while trading off intelligence.
  - **low**: Reserve for short, scoped tasks and latency-sensitive workloads that are not intelligence-sensitive.

Prompts: Calibrating Effort and Thinking Depth - 2

- **If you observe shallow reasoning on complex problems, raise effort to high or xhigh rather than prompting around it. If you need to keep effort at low for latency, add targeted guidance:**
  - **Example: "This task involves multi-step reasoning. Think carefully through the problem before responding."**
- The triggering behavior for adaptive thinking is steerable. If you find the model thinking more often than you'd like add guidance to steer it.
  - Example: "Thinking adds latency and should only be used when it will meaningfully improve answer quality — typically for problems that require multi-step reasoning. When in doubt, respond directly."

Prompts: Calibrating Effort and Thinking Depth - 3

- Conversely, if you're running hard workloads at medium and seeing under-thinking, the first lever is to raise effort. If you need finer control, prompt for it directly.

Prompts: Interactive Coding Products

- Claude tends to use more tokens in interactive settings, primarily because it reasons more after user turns.
- This can improve long-horizon coherence, instruction following, and coding capabilities in long, interactive coding sessions, but also comes with more token usage.
- To maximize both performance and token efficiency in coding products, we recommend using xhigh or high effort, adding autonomous features like an auto mode, and reducing the number of human interactions required from your users.
- Providing well-specified, clear, and accurate task descriptions upfront can help maximize autonomy and intelligence while minimizing extra token usage after user turns.

Prompts: Code Review (Bug Finding)

- Claude recommended prompt language:
  - “Report every issue you find, including ones you are uncertain about or consider low-severity. Do not filter for importance or confidence at this stage - a separate verification step will do that. Your goal here is coverage: it is better to surface a finding that later gets filtered out than to silently drop a real bug. For each finding, include your confidence level and an estimated severity so a downstream filter can rank them.”
- Be specific about language to find bugs:
- For example,
  - "report any bugs that could cause incorrect behavior, a test failure, or a misleading result; only omit nits like pure style or naming preferences."

Prompts: General Principles

- **Be clear and direct**
  - Claude responds well to clear, explicit instructions. Being specific about your desired output can help enhance results.
- Think of Claude as a brilliant but new employee who lacks context on your norms and workflows. The more precisely you explain what you want, the better the result.
- **Golden rule:**
  - Show your prompt to a colleague with minimal context on the task and ask them to follow it. If they'd be confused, Claude will be too.
  - Be specific about the desired output format and constraints.
  - Provide instructions as sequential steps using numbered lists or bullet points when the order or completeness of steps matters.
  - Example: “Create an analytics dashboard. Include as many relevant features and interactions as possible. Go beyond the basics to create a fully-featured implementation.”

Prompts: General Principles

- **Add context to improve performance**
- Providing context or motivation behind your instructions, such as explaining to Claude why such behavior is important, can help Claude better understand your goals and deliver more targeted responses.
- Example: “Your response will be read aloud by a text-to-speech engine, so never use ellipses since the text-to-speech engine will not know how to pronounce them.”

Prompts: General Principles

- **Use examples effectively**
- Examples are one of the most reliable ways to steer Claude's output format, tone, and structure. A few well-crafted examples (known as few-shot or multishot prompting) can dramatically improve accuracy and consistency.
- When adding examples, make them:
  - **Relevant:** Mirror your actual use case closely.
  - **Diverse:** Cover edge cases and vary enough that Claude doesn't pick up unintended patterns.
  - **Structured:** Wrap examples in <example> tags (multiple examples in <examples> tags) so Claude can distinguish them from instructions.

Prompts: General Principles

- **Structure prompts with XML tags**
- XML tags help Claude parse complex prompts unambiguously, especially when your prompt mixes instructions, context, examples, and variable inputs. Wrapping each type of content in its own tag (e.g. <instructions>, <context>, <input>) reduces misinterpretation.
- Best practices:
  - Use consistent, descriptive tag names across your prompts.
  - Nest tags when content has a natural hierarchy (documents inside <documents>, each inside <document index="n">).

Prompts: General Principles

- **Long context prompting**
- When working with large documents or data-rich inputs (20k+ tokens), structure your prompt carefully to get the best results:
- **Put longform data at the top:** Place your long documents and inputs near the top of your prompt, above your query, instructions, and examples. This can significantly improve performance across all models.
- **Structure document content and metadata with XML tags:** When using multiple documents, wrap each document in <document> tags with <document_content> and <source> (and other metadata) subtags for clarity.
- **Ground responses in quotes:** For long document tasks, ask Claude to quote relevant parts of the documents first before carrying out its task. This helps Claude cut through the noise of the rest of the document's contents.

Bad Prompt vs Good Prompt

- Prompts need to be specific, not vague

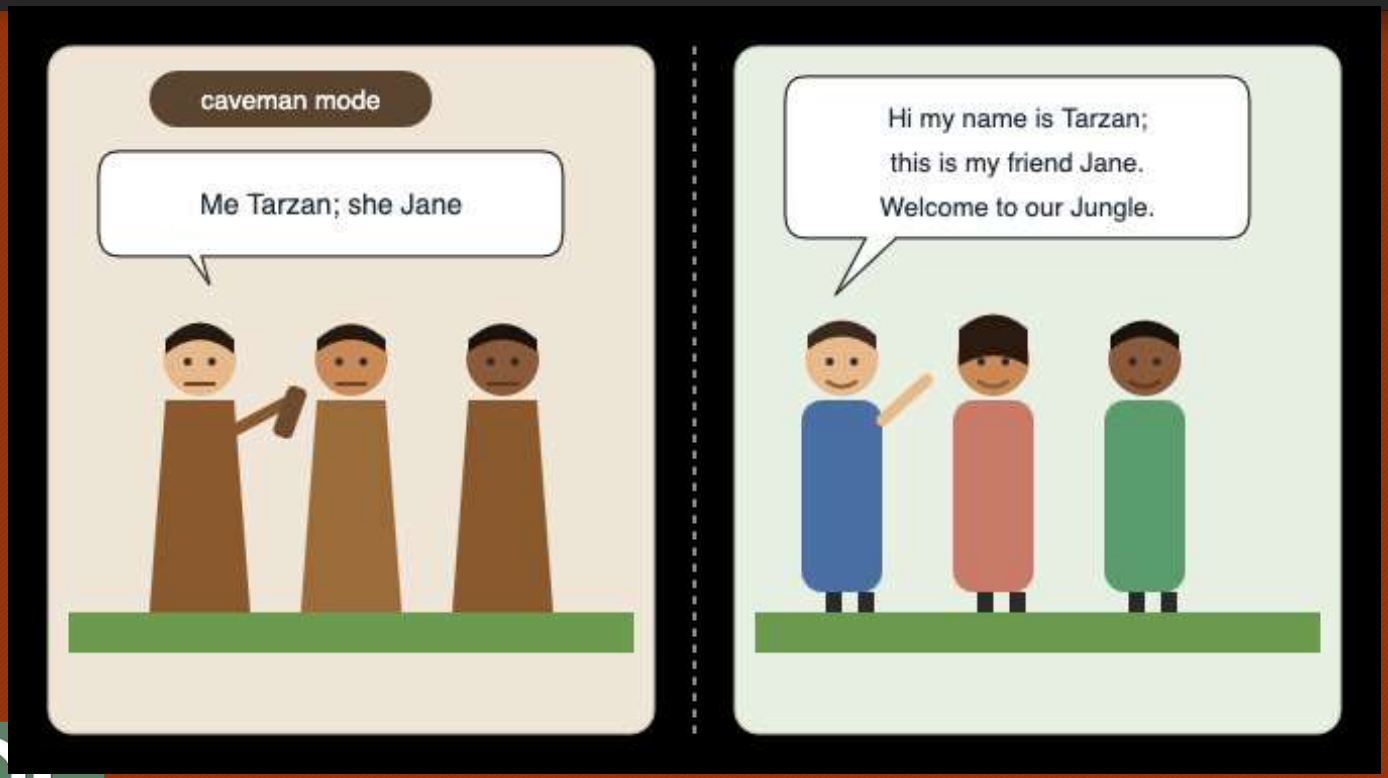
- A bad prompt:
- Give me a report on Abraham Lincoln

- A better version:
- Give me a 5 page report on Abraham Lincoln
- Include:
  - Early years as a lawyer
  - Why he decided to run for President
  - His campaign struggles
  - Summary of his failures and successes in each of the years of his Presidency
  - How historians regard him as a President
  - Do not have any section over weighted compared to the others
- Provide a second one page list of suggested additional topics to be included in the report

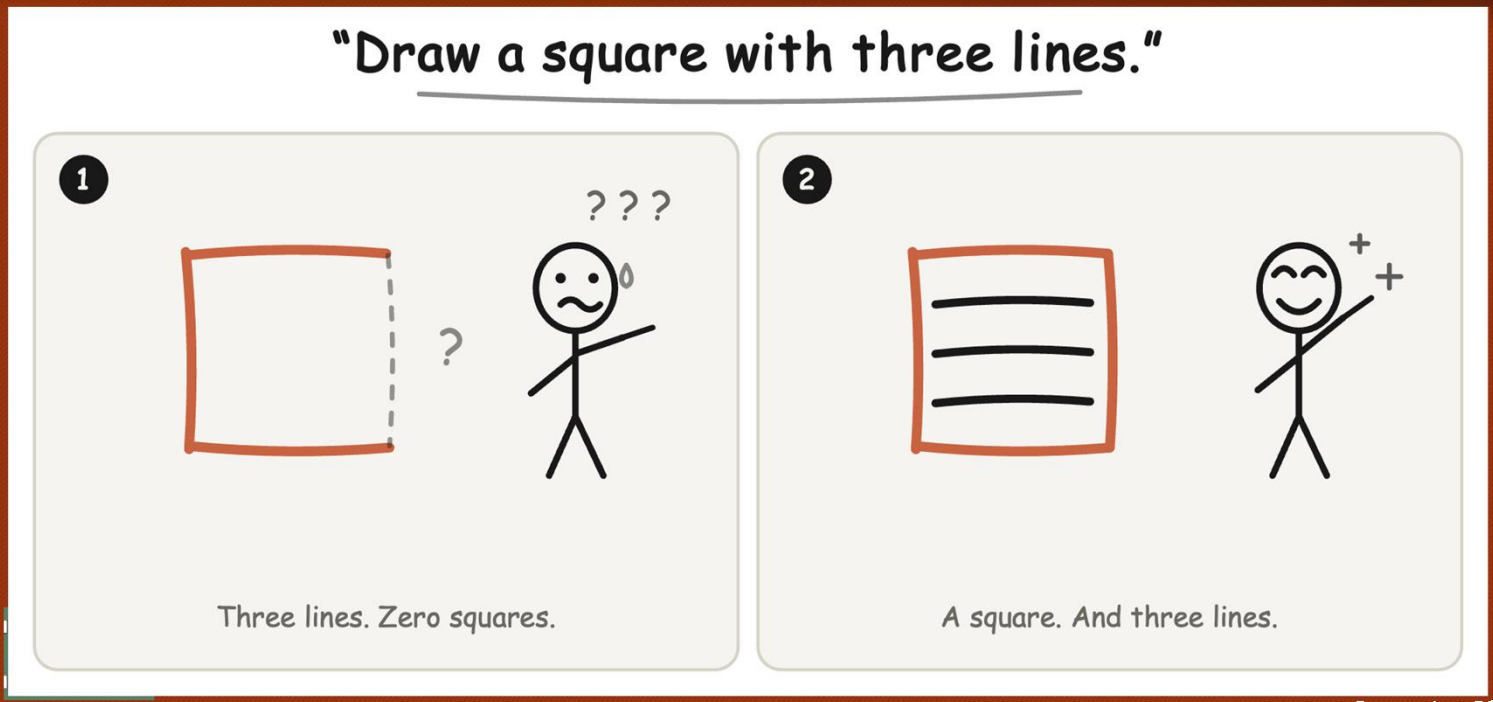
Caveman Mode

- Caveman mode is a prompting trick where you tell Claude to drop everything but core meaning:
  - no articles (a, an, the), no filler words (just, really, basically),
  - no pleasantries ("Great question!"),
  - no restating the problem back to you, fragments allowed.
  - Output ends up terse, almost broken-English, but dense with information.
  - The payoff is token savings, often 60-75% fewer output tokens, and faster reads since there's nothing to skim past.
- Best used for:
  - quick coding answers,
  - terse status checks,
  - debugging output,
  - anything where you want the raw fact or fix and nothing else.
- Worst fit: explanations,
  - teaching material,
  - anything nuanced where you actually need connecting words and hedges to convey how confident or conditional something is.
  - Compressing those to fragments tends to lose the meaning you needed.

Caveman Mode to Save Tokens



How Prompts can be Misinterpreted



More of How Prompts can be Misinterpreted

- Asked the AI to make my code more readable.
  - It renamed every variable to theVariable, theOtherVariable, theThirdVariable …
- I asked for a function that never fails.
  - It returned void and did nothing.
  - Zero failures.
- Told it to remove all the bugs.
  - It deleted the tests.
- Asked it to summarize a 40-page doc in one sentence.
  - It used semicolons; twelve of them.

Claude Skills

- Skills are capabilities that extend Claude's functionality
- Skill require a specific format
- Each Skill packages:
  - Instructions,
  - Metadata, and
  - Optional resources (scripts, templates),

  that Claude uses automatically when relevant.
- Skills are reusable, filesystem-based resources that provide Claude with domain-specific expertise:
  - Workflows,
  - Context, and
  - Best practices,

  that transform general-purpose agents into specialists.

Why Use Skills

- When actions are repeated often, skills provide a repeatable workflow
- Examples:
  - All presentations have a common look and feel for an individual/company
  - All documentation has a common look and feel for an individual/company
- Claude has a set of built-in skills
- Users can create their own skills

Some Claude Built-in skills

Skill	Description
skill-creator	Used to create user skills
docx	Create word file
pdf	Create pdf from other source
pptx	Create power point file
xlsx	Create excel file
frontend-design	Create websites
mcp-builder	Create MCP (Model Context Protocol) Servers

https://github.com/anthropics/skills/tree/main/skills

User Created Skills

- Using the skill-creator skill, new skills can be created
- Example: fsd-creator
- Input a high-level description of a project and create a compete FSD with:
  - numbered requirements,
  - numbered tests, and
  - any other material needed for Claude to write the code
- Example: code-creator skill
- Read an FSD.md document and write and test code in a specified language

Skill Structure

- Every Skill requires a SKILL.md file with YAML frontmatter
- Required fields: name and description
- Field requirements:
- name:
  - Maximum 64 characters
  - Must contain only lowercase letters, numbers, and hyphens
  - Cannot contain XML tags
  - Cannot contain reserved words: "anthropic", "claude"
- description:
  - Must be non-empty
  - Maximum 1024 characters
  - Cannot contain XML tags
- The description should include both what the Skill does and when Claude should use it. For complete authoring guidance, see the best practices guide.

```
---
name: your-skill-name
description: Brief description of what this
Skill does and when to use it
---

# Your Skill Name

## Instructions
[Clear, step-by-step guidance for Claude
to follow]

## Examples
[Concrete examples of using this Skill]
```

https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices

The Human Developer vs The AI Developer

- What is the difference between a Human with a PhD in computer science and an AI system?
- The PhD spent six years and a small fortune to learn things the AI absorbed over a weekend of training runs.
- But the PhD can at least explain why the code doesn't work, while the AI just confidently ships it anyway.

Claude Tools

- A tool is a single capability the AI Assistant can invoke — with
  - A name,
  - A description, and
  - A defined set of parameters
  - The description tells the model when to use it; the schema tells it how
- A tool is something the **AI** invokes, not something the user runs
- Most tools are provided by:
  - The platform (read, bash, web_search) or
  - By an MCP server someone else wrote
- Most tools aren't user-built. Building your own is one option
  - (but Claude thinks users don’t build tools)
- A tool is a *verb the AI can execute*
- A skill is *instructions telling the AI how to do a job*, which may involve several tools

WHAT IS A TOOL?

- A tool is three things:
  - a name,
  - a plain-English description, and
  - a JSON Schema for its inputs.
- Claude never executes anything.
  - It emits a structured request;
  - your code runs it.
- The description is the prompt.
  - It is how Claude decides when — and when not — to call.
- Default tool_choice is "auto":
  - Claude decides each turn.
  - Use "any", "tool", or "none" to force the issue.

- New on Fable 5: thinking is always on.
  - Echo the thinking blocks back unchanged with every tool_result.
- Tune the loop with output_config effort —
  - low…max.
  - Lower effort means fewer tool calls; higher means more exploration.
- Same mechanism whether the tool is
  - an SQL query,
  - a GPIO write, or
  - a Jira ticket.

DEFINING YOUR OWN TOOL


```
tools = [{
  "name": "get_weather",
  "description": (
      "Get current weather for a location. Use when the user asks about "
      "weather, temperature, or sky conditions. Returns temperature in "
      "Celsius plus a short sky state. Does NOT return forecasts."
  ),
  "input_schema": {
      "type": "object",
      "properties": {
          "location": {"type": "string",
                       "description": "City and state, e.g. Boston, MA"}
      },
      "required": ["location"]
  }
}]

resp = client.messages.create(model="claude-opus-5", tools=tools,
                              max_tokens=1024, messages=msgs)
```

Rule of thumb: 3–4 sentences per description — what it does, when NOT to use it, what it does not return.

TOOLS ANTHROPIC ALREADY PROVIDES

CLIENT TOOLS — you execute

Anthropic publishes the schema and trains Claude on it. You still run the call and return a tool_result.

`bash` shell commands, persistent session
`text_editor` view / edit files
`memory` state across conversations
`computer` screenshots, mouse, keyboard

Priced like any other request — you only pay for tokens.

SERVER TOOLS — Anthropic executes

Run on Anthropic's infrastructure. No handler code — results come back in the same response.

`web_search` cited results past the cutoff
`web_fetch` full page and PDF content
`code_execution` sandboxed Python / bash
`tool_search` load from thousands of tools on demand

Some add usage-based charges on top of tokens — e.g. per search performed.

TOOL vs. SKILL

A TOOL IS A CAPABILITY

A function you register in tools[]: name, description, JSON schema.

- Adds something Claude physically could not do before.
- Claude emits tool_use; your code executes it.
- Full schema sits in context on every request.
- Code you write, maintain, and secure.

A SKILL IS A PROCEDURE

A folder: SKILL.md plus optional scripts, templates, references.

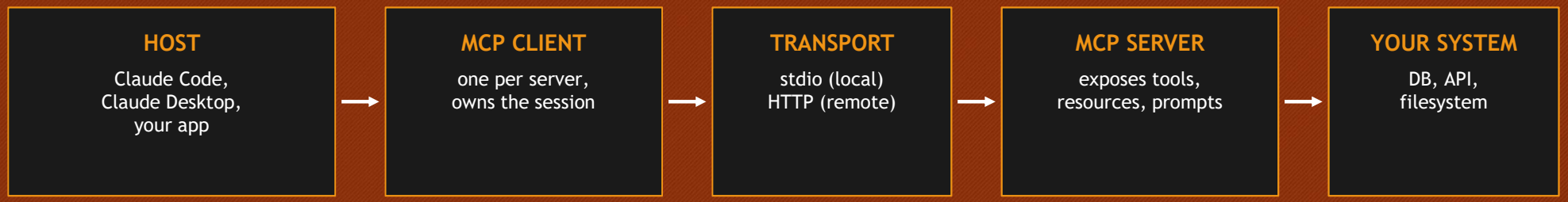
- Adds no new capability — teaches how to use what exists.
- Ships scripts that run through existing tools, e.g. bash.
- Loaded on demand — only name + description stay resident.
- Mostly Markdown — a non-engineer can write one.

A tool is a syscall. A skill is the man page — plus the wrapper script your senior dev wrote so you call it correctly.

MCP — WHY IT EXISTS

- Every agent you build needs the same integrations: Jira, Postgres, Git, the wiki.
- Without a standard, you rewrite each one per app, per language, per framework.
- MCP is an open protocol for exposing tools over a format/protocol any client can speak.
- Write the integration once as a server; every MCP-aware client gets it for free.

HOW MCP FITS TOGETHER



- The host runs a client per server; the client does discovery, then relays tool calls.
- Servers advertise tools, resources, and prompts. Claude's MCP connector supports tool calls today.
- stdio for a server on the same machine; Streamable HTTP or SSE for anything remote.

CONNECTING TO AN MCP SERVER


```
resp = client.beta.messages.create(
    model="claude-opus-5", max_tokens=1000,
    messages=[{"role": "user", "content": "What's blocking the release?"}],
    mcp_servers=[{
        "type": "url", "name": "jira",
        "url": "https://mcp.example.com/sse",
        "authorization_token": OAUTH_TOKEN,     # you run the OAuth flow
    }],
    tools=[{
        "type": "mcp_toolset", "mcp_server_name": "jira",
        "configs": {"delete_issue": {"enabled": False}},   # denylist writes
    }],
    betas=["mcp-client-2025-11-20"],
)
```

- The connector needs a public https:// server — local stdio servers cannot attach directly.
- Tool calls only for now; use the SDK's MCP helpers for local servers, prompts, or resources.
- Denylist destructive tools. For servers with dozens of tools, set defer_loading and add tool search.

BUILDING YOUR OWN MCP SERVER


```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("build-tools")

@mcp.tool()
def get_build_status(branch: str = "main") -> str:
    """Return CI status for a branch.

    Use when asked whether a build passed,
    failed, or is still running.
    """
    return ci.status(branch)

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

The docstring becomes the tool description; type hints become the JSON schema.

SERVER, OR JUST A TOOL?

Build a server when…

- more than one app or agent needs the capability
- another team owns the system behind it
- you want it in Claude Code, the desktop app, and your API service at once

Skip it when…

- one app, one codebase, one deploy — a plain tool definition is less machinery

WHICH ONE DO I REACH FOR?

SKILL

Claude can already do it — just not the way you want.

- The capability exists; the procedure doesn't.
- House style, review checklists, recurring workflows.
- Markdown plus optional scripts — anyone can author one.
- Here: the CDL brand rules that styled this deck.

TOOL

Claude cannot reach the thing at all.

- A new capability: your DB, an internal API, hardware.
- One app, one codebase, one deploy.
- You own the schema, the execution, and the auth.
- Here: get_build_status hitting your CI server.

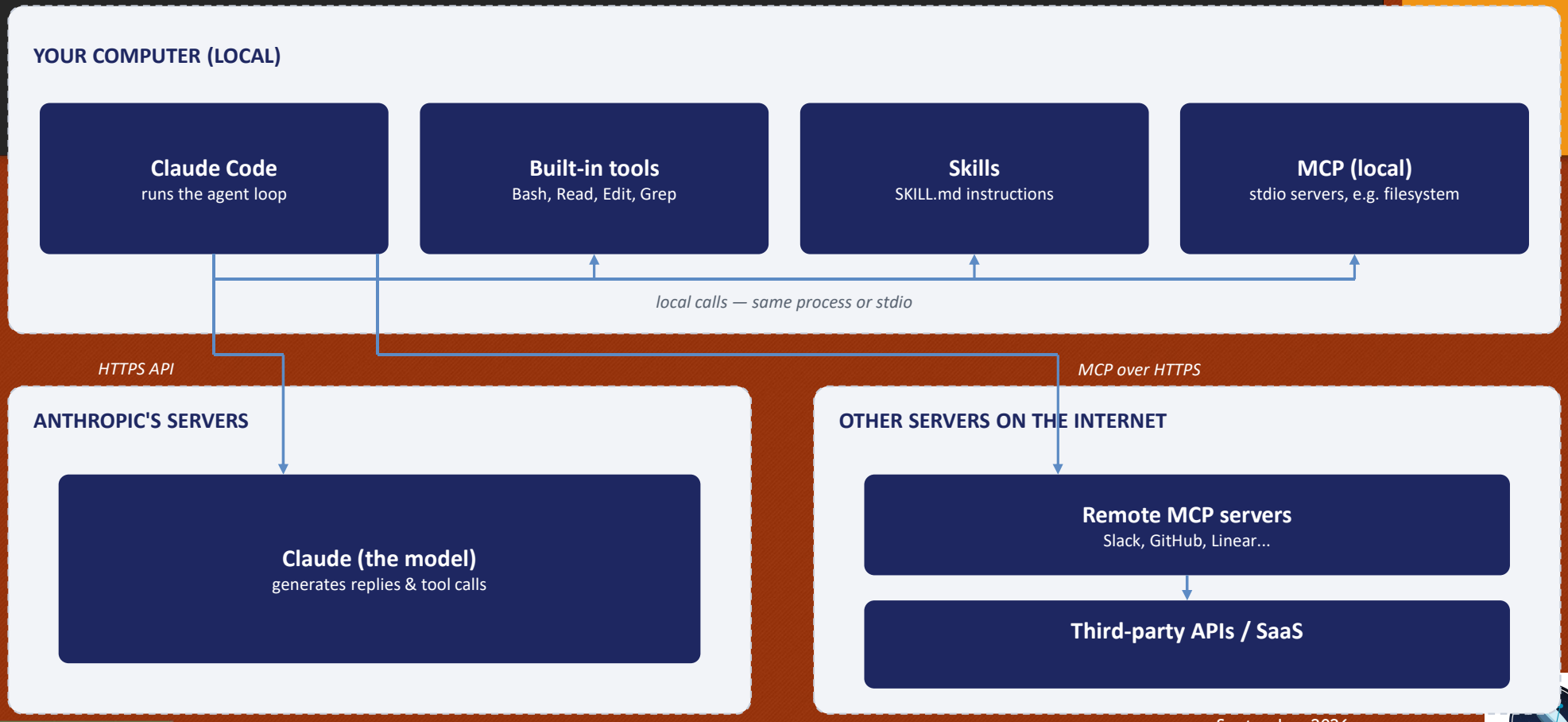
MCP

More than one client needs that same capability.

- Claude Code, the desktop app, and your service at once.
- Another team owns the system; ship and version it apart.
- Open protocol — non-Claude clients get it too.
- Here: one Jira server the whole org attaches to.

Already possible? Skill. **Needs a new capability?** Tool. **Needed by more than one client?** MCP.

Claude Code Architecture



September 2026

Claude created this graphic: skills can also exist on Anthropic Servers

The Types of Claude at a Glance

claude.ai WEB CHAT
Browser tab. Turn-by-turn conversation. Uploads, projects, connectors.
Nothing executes on your machine.

DESKTOP APP WINDOWS / macOS
Same account, same synced chats.
Adds local filesystem reach and local MCP servers a browser tab cannot host.

CLAUDE CODE CLI + IDE
Terminal, VS Code, JetBrains.
Reads the repo, edits files, runs builds and tests, commits, opens PRs.

COWORK AGENT, NO TERMINAL
Same engine as Code, aimed off-repo.
Files, data, research; ships .xlsx / .docx / .pptx deliverables.

One model family throughout. What changes is the shell — and whether the work is code.

CHAT: CLAUDE.AI VS. THE DESKTOP APP

claude.ai — WEB

- Runs in any browser, nothing to install
- Turn-by-turn: you ask, it answers
- Uploads, Projects, remote connectors
- Conversations sync to every other surface

DESKTOP APP — WINDOWS / macOS

- Same account, same conversations
- Reaches files on your local disk
- Hosts locally-installed MCP servers
- Is the shell Cowork runs inside

Not two products — one chat product in two shells. The desktop shell exists for local access.

CLAUDE CODE: THE ENGINEERING SURFACE

- Terminal CLI, plus VS Code and JetBrains extensions
- Scope: software engineering — repo in, working tree out
- Reads source, edits files, runs builds and tests
- Commits, pushes, opens pull requests
- Agentic loop: give a goal, it iterates until met
- Not snippets to paste — it edits your checkout
- Drives your toolchain: make, CMake, MSBuild, pytest

```
$ claude
> fix the off-by-one in ring_buffer.c

  * Read   src/ring_buffer.c
  * Edit   src/ring_buffer.c   (+3 -2)
  * Bash   cmake --build build
  * Bash   ctest --output-on-failure
           12/12 tests passed
  * Bash   git commit -m "fix: rb wrap"

  Done. 1 file changed, tests green.
```

COWORK: SAME ENGINE, DIFFERENT TARGET

- Same agentic architecture as Claude Code — no terminal
- Target is knowledge work, not repositories
- Describe an outcome, then step away while it works
- Runs code in a sandbox; bridges to your local folders
- Delivers finished files: .xlsx, .docx, .pptx, .pdf
- Schedules recurring tasks — reports, digests
- Same chat account, same conversation history

WHERE COWORK RUNS

```
Desktop, macOS / Windows ..... all paid plans
Web, claude.ai ............... Pro, Max, Team
Mobile, iOS / Android ........ Pro, Max, Team
Chrome side panel ............ Max, Team
                               (Pro rolling out)
Free plan .................... not included

Enterprise: web + mobile need admin enablement.
```

This deck was built by Cowork.

WHICH DO I USE WHEN?

Repo, build, test, commit, review	→	**CLAUDE CODE**
Files, data, research, reports, decks	→	**COWORK**
Think out loud, one-off questions	→	**CHAT (web or desktop)**
Anything that must touch your local disk	→	**DESKTOP APP (or a Cowork folder)**

Code and Cowork keep working on their own. Chat waits for you every turn.

Claude Code for Software Development

100

We will now describe how to install Claude code

And we will be using it for the rest of the class

Installing Claude within a Docker Container - 1

- Anthropic has provided a Docker Container for Claude Code
- https://github.com/anthropics/claude-code/tree/main
- Start Power Shell
- Run wsl.exe
- If git is not already installed, run:
  - sudo add-apt-repository ppa:git-core/ppa
  - sudo apt-get update
  - sudo apt-get install git

Installing Claude within a Docker Container - 2

- cd /"name_of_directory_where_you_want_to_install_claude"
- sudo git clone https://github.com/anthropics/claude-code
- cd .devcontainer
- nano devcontainer.json
- Add to line "postStartCommand … firewall.sh||true before the ,
- Scroll up to the end of the mounts section

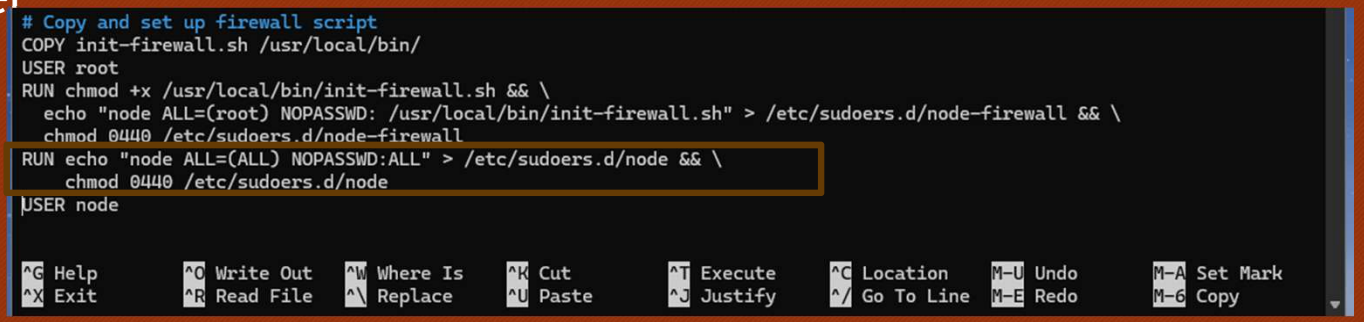
```
},
"workspaceMount": "source=${localWorkspaceFolder},target=/workspace,type=bind,consistency=delegated",
"workspaceFolder": "/workspace",
"postStartCommand": "sudo /usr/local/bin/init-firewall.s || true",
"waitFor": "postStartCommand"
}
```

```
"mounts": [
  "source=claude-code-bashhistory-${devcontainerId},target=/commandhistory,type=volume",
  "source=claude-code-config-${devcontainerId},target=/home/node/.claude,type=volume"
],
"runArgs" : [
  "--device=/dev/ttyACM0", //esp32 serial port
],
"containerEnv": {
```

- Add the text in the dotted box to support serial ports
- Save and exit

Installing Claude within a Docker Container - 3

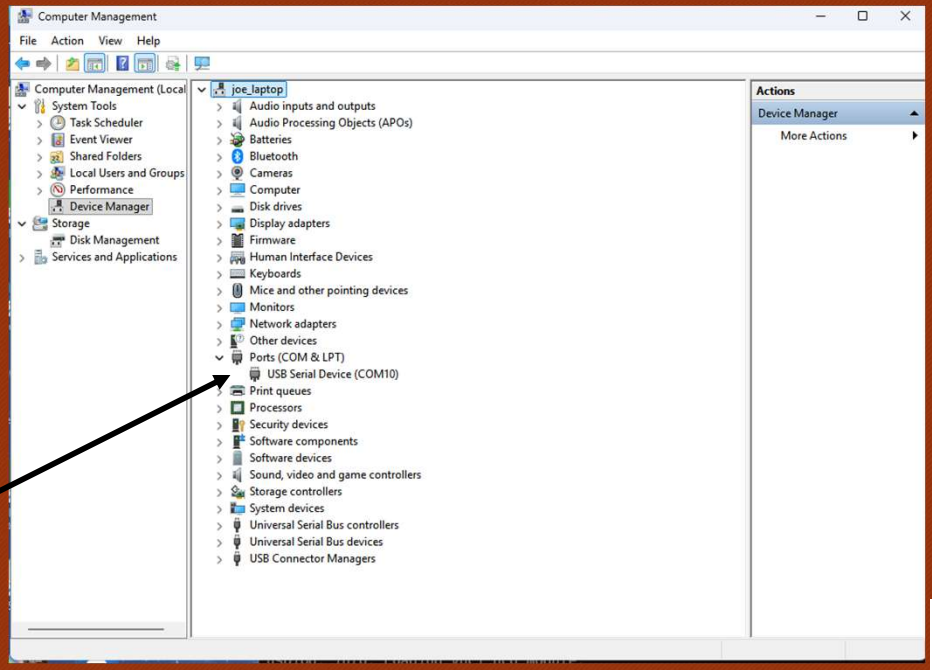
- The Docker Container needs to run some commands as superuser (sudo)
- To remove the need for a password on sudo we need to edit the Docker file In the .devcontainer directory
- Type nano Docker
- Near the end of the file add the boxed text to the file



- Save and exit

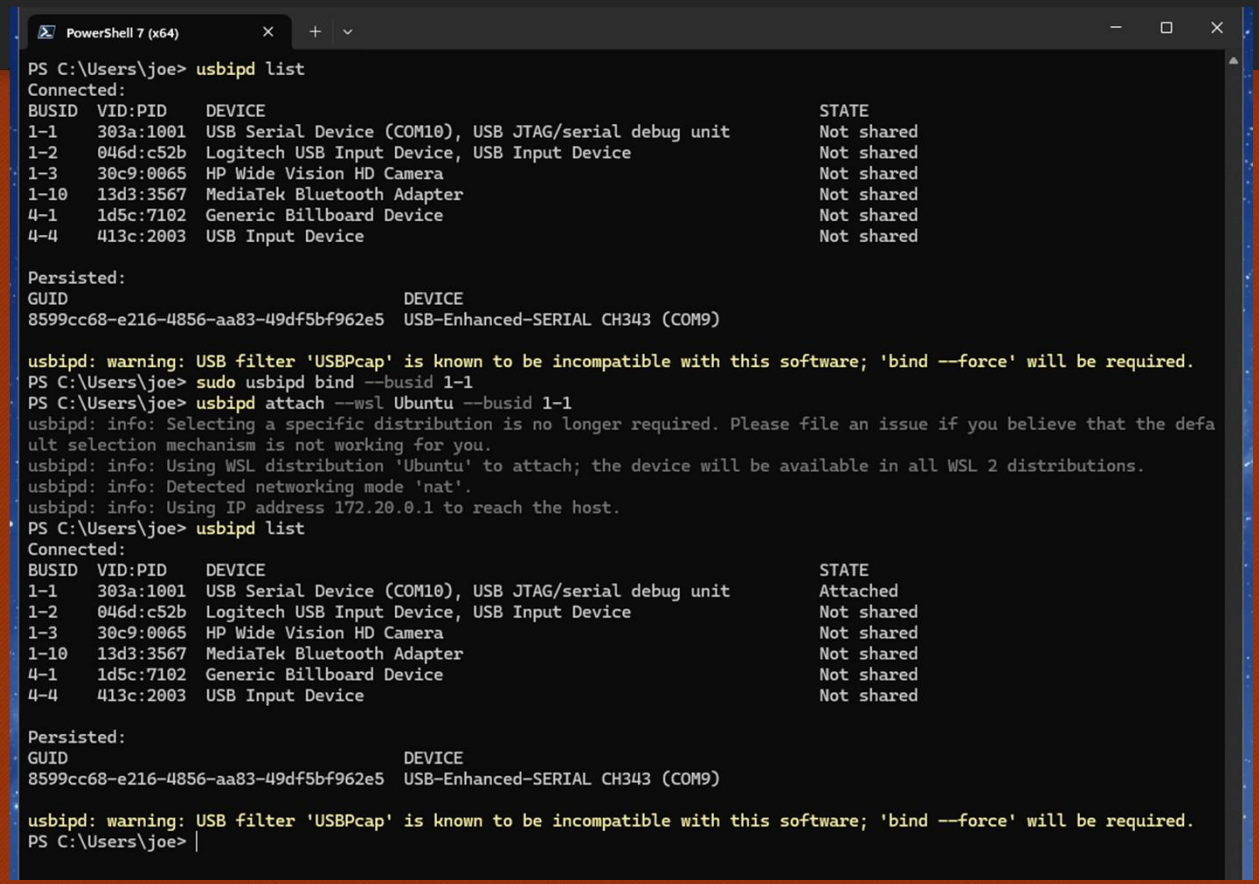
Installing Claude within a Docker Container - 4

- If using USB port on laptop to access a microprocessor, we need to edit .devfile file to tell it that it can access the USB port (/dev/ttyACM0)
- Unfortunately, if we disconnect the port, the docker file will not run
- First find the port in windows



Installing Claude within a Docker Container - 6

- Then run PowerShell and use the usbipd command to tell WSL that we have a USB port



Installing Claude within a Docker Container - 7

- Run wsl and confirm that it sees the USB port (/dev/ttyACM0)

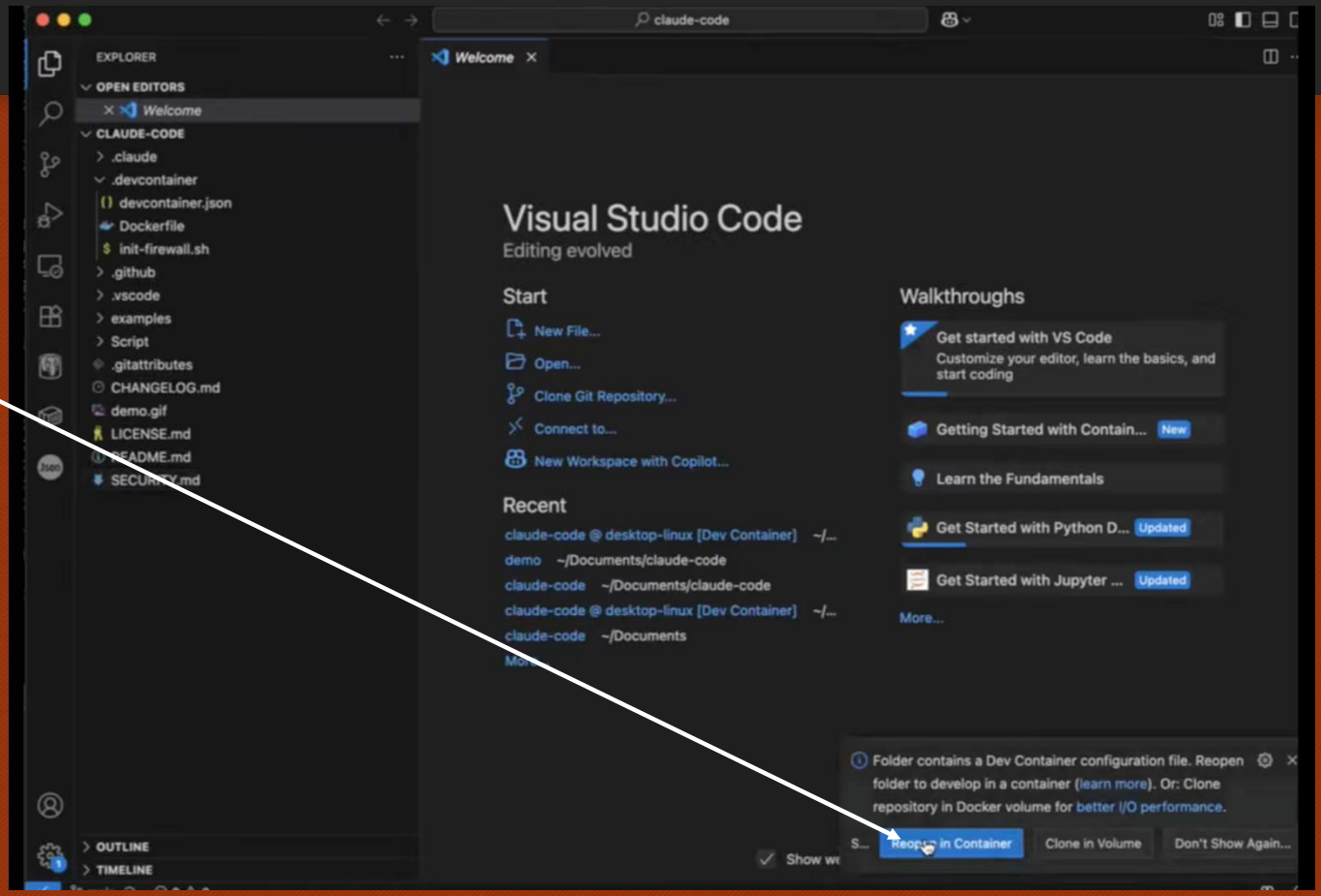
```
PS C:\Users\joe> wsl.exe
joe@joelaptop:/mnt/c/Users/joe$ ls /dev/tty*
/dev/tty    /dev/tty16  /dev/tty24  /dev/tty32  /dev/tty40  /dev/tty49  /dev/tty57  /dev/tty8     /dev/ttyS6
/dev/tty0   /dev/tty17  /dev/tty25  /dev/tty33  /dev/tty41  /dev/tty5   /dev/tty58  /dev/tty9     /dev/ttyS7
/dev/tty1   /dev/tty18  /dev/tty26  /dev/tty34  /dev/tty42  /dev/tty50  /dev/tty59  /dev/ttyACM0
/dev/tty10  /dev/tty19  /dev/tty27  /dev/tty35  /dev/tty43  /dev/tty51  /dev/tty6   /dev/ttyS0
/dev/tty11  /dev/tty2   /dev/tty28  /dev/tty36  /dev/tty44  /dev/tty52  /dev/tty60  /dev/ttyS1
/dev/tty12  /dev/tty20  /dev/tty29  /dev/tty37  /dev/tty45  /dev/tty53  /dev/tty61  /dev/ttyS2
/dev/tty13  /dev/tty21  /dev/tty3   /dev/tty38  /dev/tty46  /dev/tty54  /dev/tty62  /dev/ttyS3
/dev/tty14  /dev/tty22  /dev/tty30  /dev/tty39  /dev/tty47  /dev/tty55  /dev/tty63  /dev/ttyS4
/dev/tty15  /dev/tty23  /dev/tty31  /dev/tty4   /dev/tty48  /dev/tty56  /dev/tty7   /dev/ttyS5
joe@joelaptop:/mnt/c/Users/joe$
```

Installing Claude within a Docker Container - 8

- We can now build the Docker Container
- If not running click on docker desktop icon
- Get back to the claude_code directory
- Type: code .
- Visual Studio will run
- Click on the box in lower right “Reopen in Container”
- The Docker Container will be built

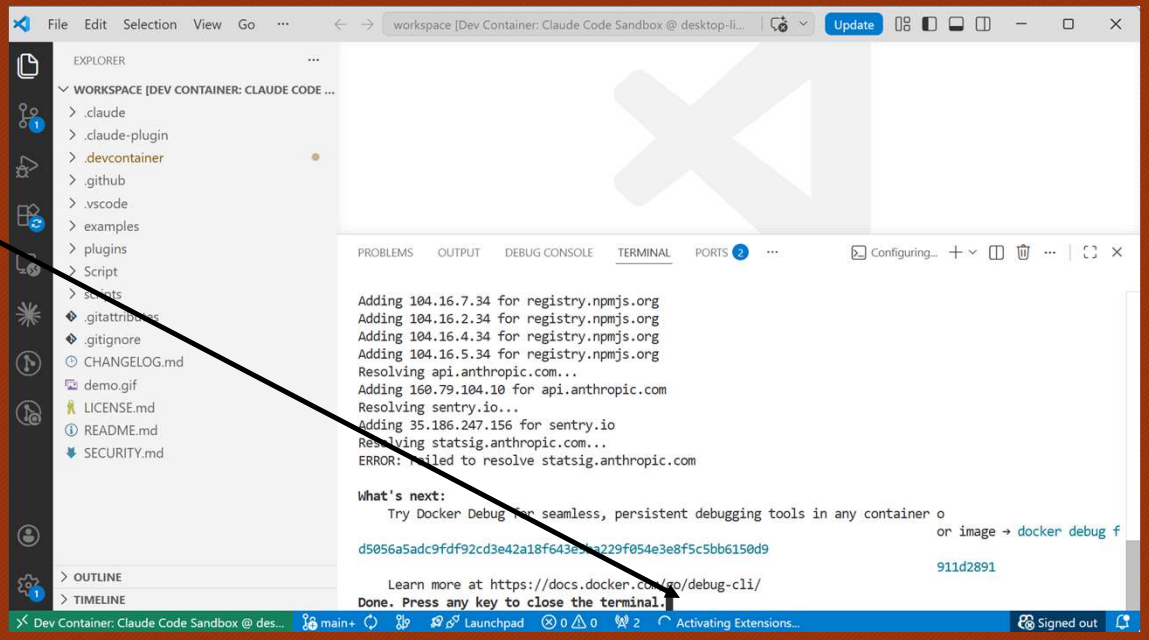
Installing Claude within a Docker Container - 9

Click Here



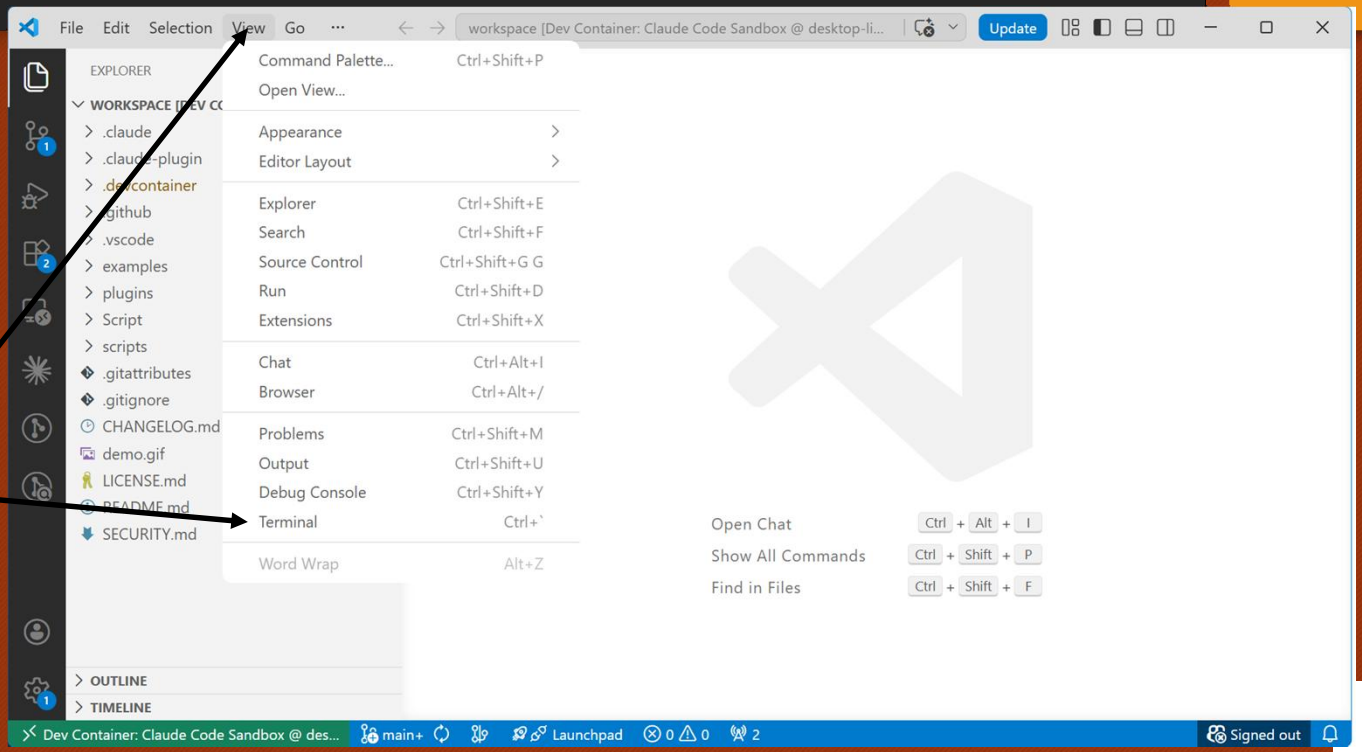
Installing Claude within a Docker Container - 10

Hit any key



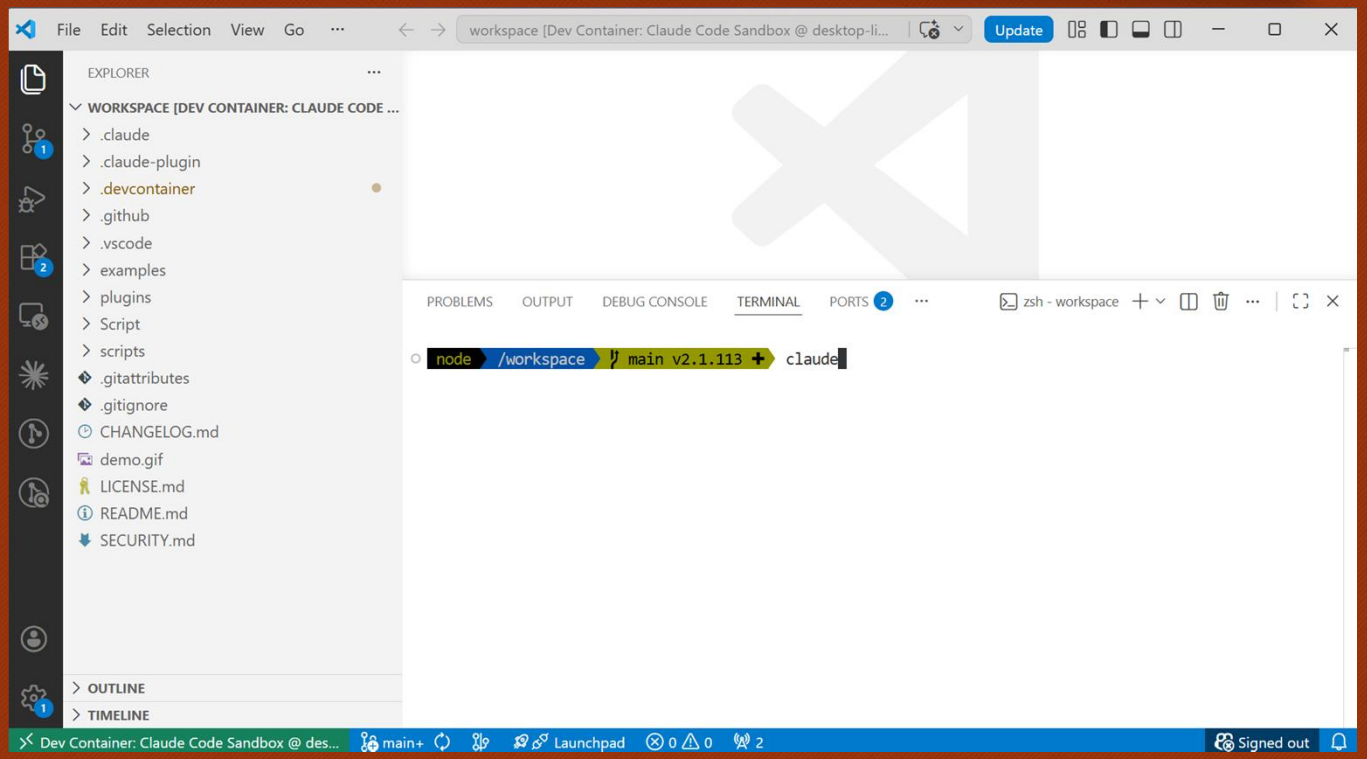
Installing Claude within a Docker Container - 11

- After the container is built and the Terminal is closed, open a new terminal by clicking on View and Terminal



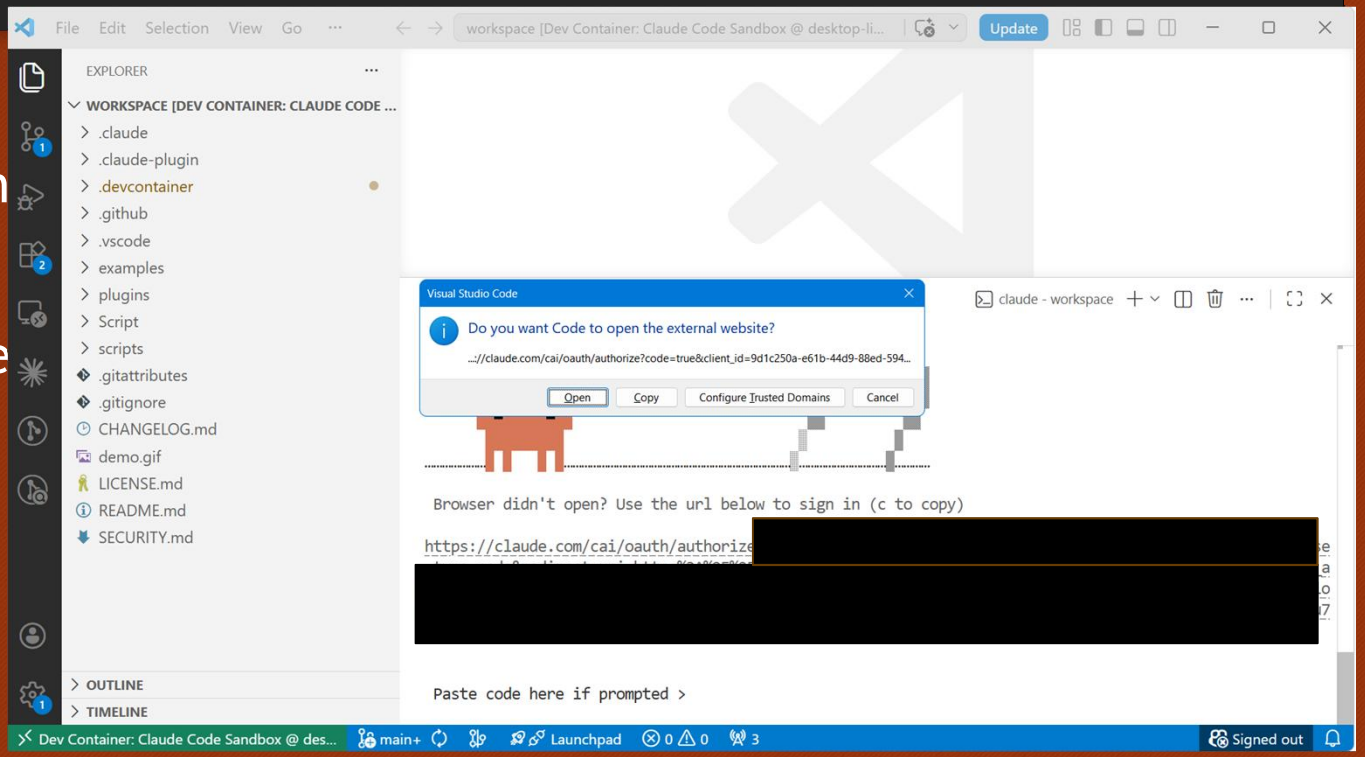
Installing Claude within a Docker Container - 12

- In the Terminal window type: claude



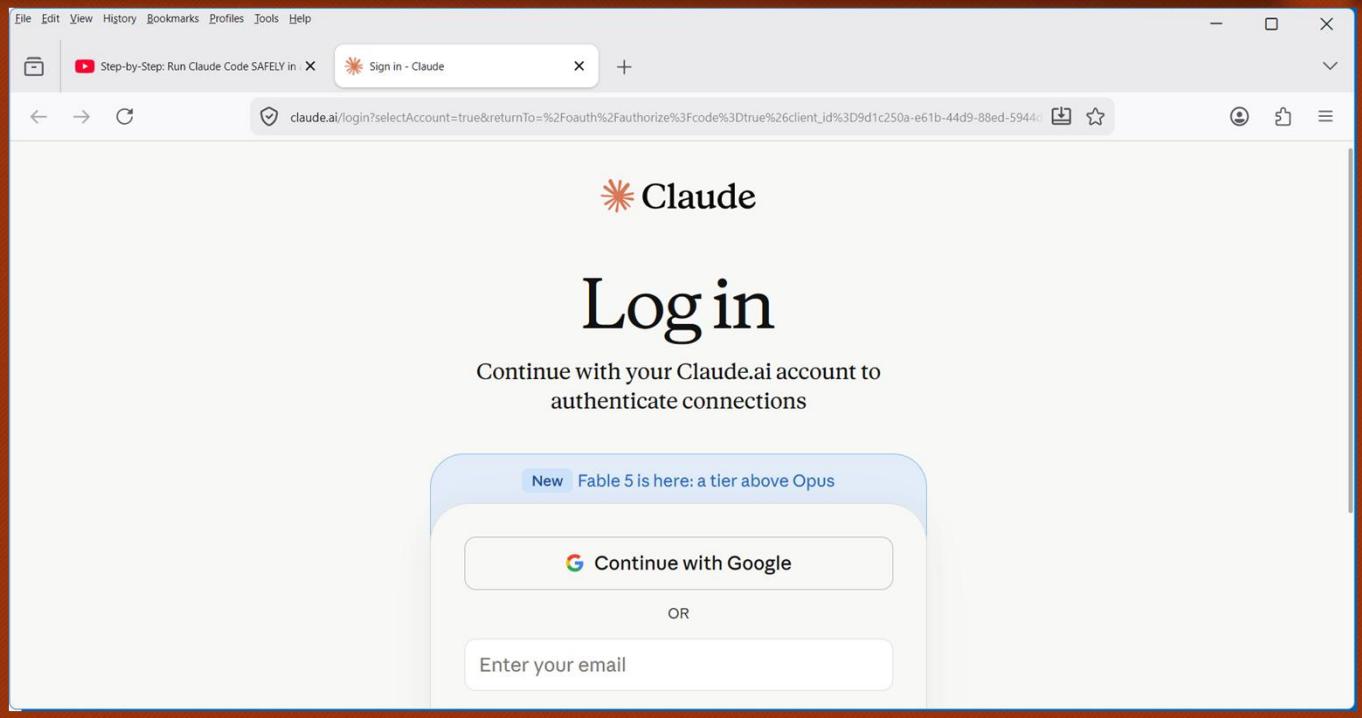
Installing Claude within a Docker Container - 13

- The first time you run claude you must authenticate with Anthropic



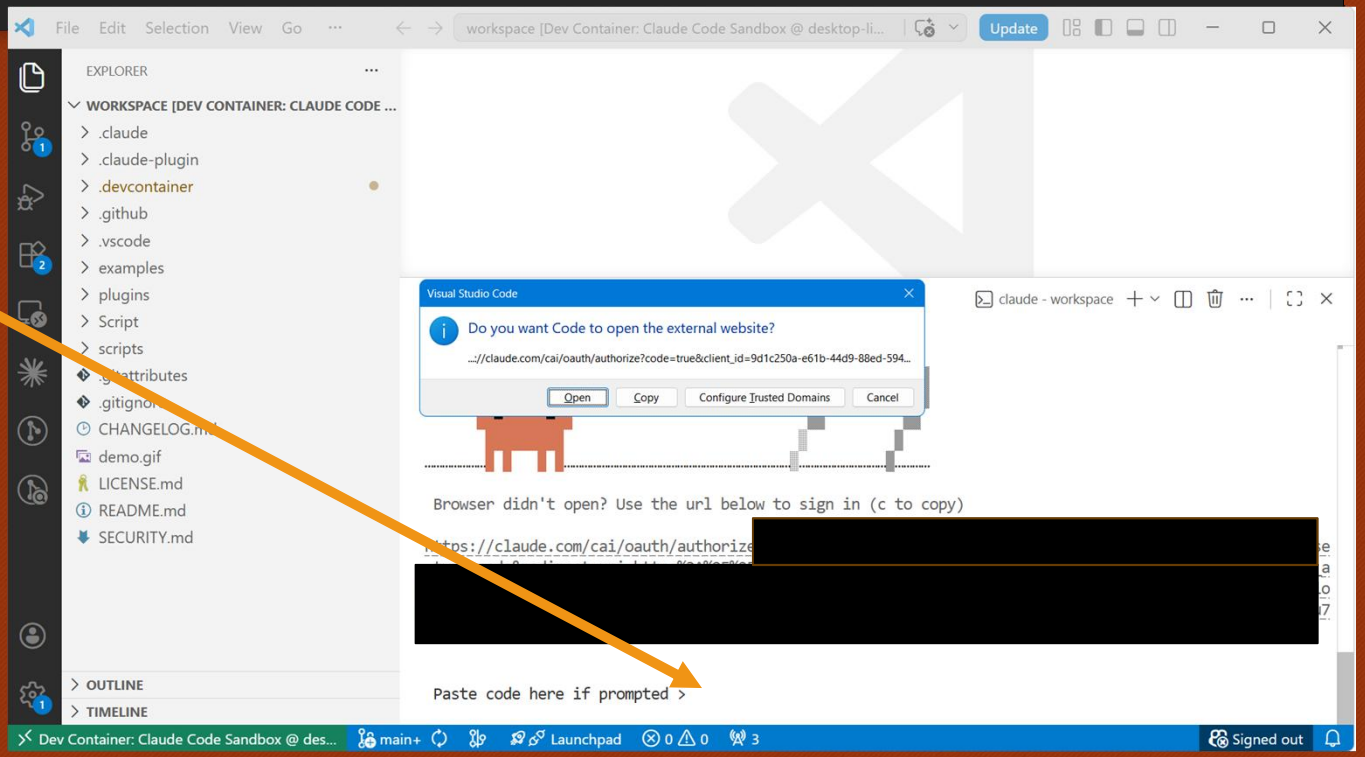
Installing Claude within a Docker Container - 14

- The first time you run Claude you must authenticate with Anthropic
- After you login you will see a random set of characters to copy and past into Claude Code



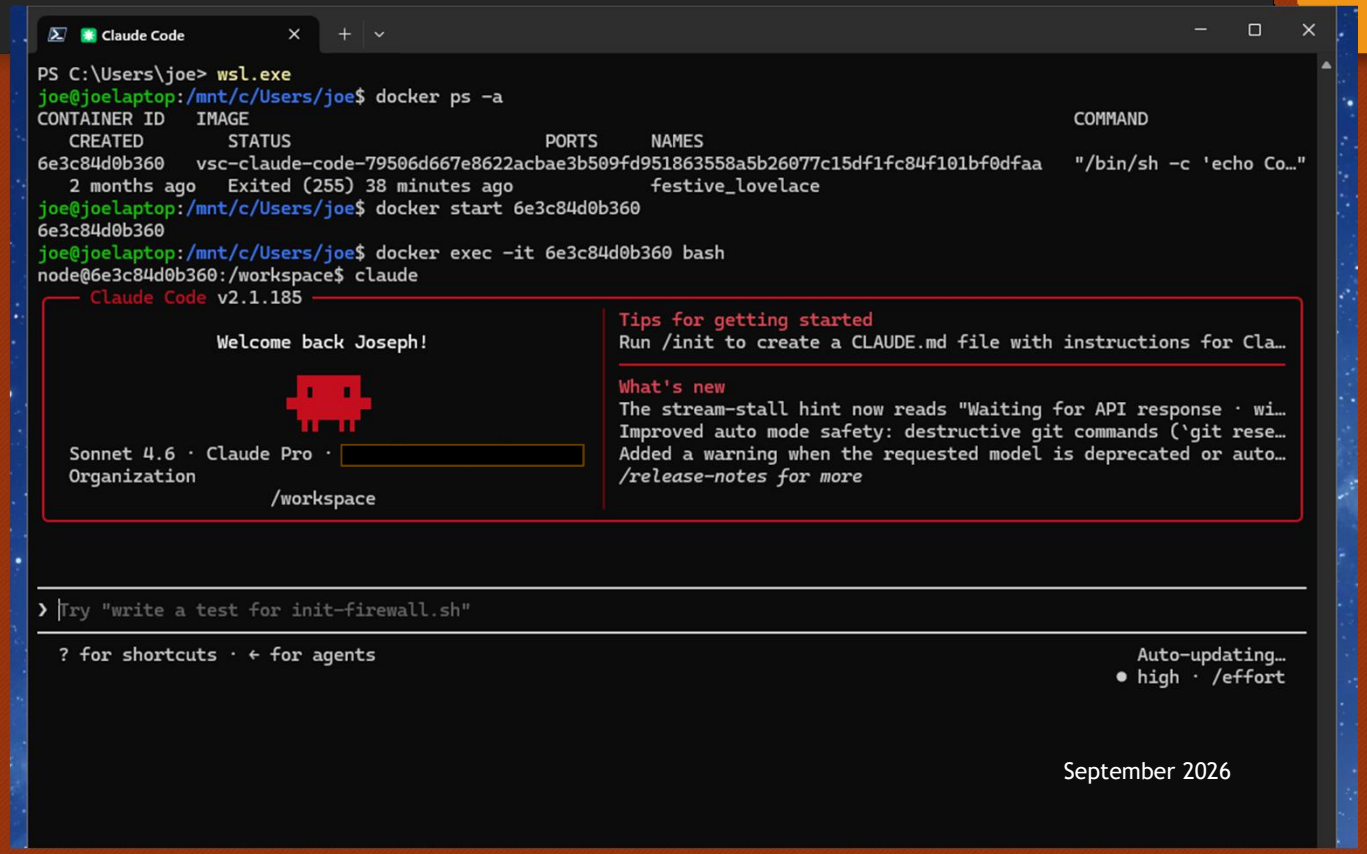
Installing Claude within a Docker Container - 13

- Past the characters here



Installing Claude within a Docker Container - 15

- If you do not want to run Visual Studio Code, you can start the Container with the following commands



September 2026

Using VS Code for Development

- When Anthropic's Claude Code is installed in a container using Anthropic's container code, it automatically supports Visual Studio Code
- Thus, developers do not need to install it separately

Using the Arduino CLI for Development

- Originally the Arduino IDE was developed to simplify programming of the Arduino Microprocessor board
- Rather than writing to registers to set parameters of the microprocessor, the Arduino IDE using command like
  - pinMode(LED_BUILTIN, OUTPUT);
- Many microprocessor boards are now supported by the Arduino IDE
- A command line interface (CLI) can be installed in the Container for development

Homework

- Install Anthropic's Docker Container on your laptop
- If you have not chosen a class project, choose one
- Use the skill-creator skill to:
  - Create your own FSD-creator skill that you will use for your project
  - Have the skill prompt you for specifics for a project
- Start Creating the FSD for your project using the skill you just created

Questions?

Lab work

- Start/Continue working on your class project
- Instructors will help with any problems